

프레임워크 개발자 가이드

기본 셋업이 끝난 뒤, 업무 코드를 작성할 때 바로 찾아 쓰는 실무 매뉴얼

공식 문서 02

기준: 최신 로컬 Source · 처음 사용하는 사용자 관점

전체 목차

필요한 항목을 누르면 해당 장으로 바로 이동합니다.

- 1 [업무 개발 Quick Finder](#)
- 2 [자주 쓰는 명령과 확인 위치](#)
- 3 [Controller 작성](#)
- 4 [Service 작성](#)
- 5 [DB·Repository·Paging](#)
- 6 [Transaction](#)
- 7 [Cache·Lock](#)
- 8 [공통코드·파라미터·메시지·영업일](#)
- 9 [Logging·Context·Audit](#)
- 10 [외부 연계](#)
- 11 [다른 Domain 호출](#)
- 12 [Messaging](#)
- 13 [Security·권한·승인](#)
- 14 [Test·오류 확인](#)
- 15 [필요 기능을 추가할 때](#)
- 16 [빠른 Reference](#)

1. 업무 개발 Quick Finder

[전체 목차로](#)

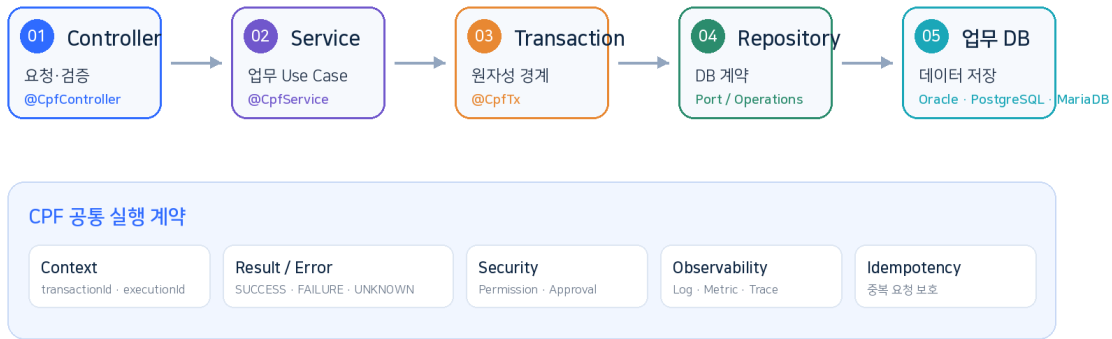
이 문서는 CPF 내부 구조를 설명하는 교재가 아니라, 기본 실행 환경이 준비된 개발자가 업무 코드를 만들면서 바로 찾아 쓰는 문서입니다. 먼저 아래 표에서 하고 싶은 일을 찾고, 필요한 장만 읽으면 됩니다.

하려는 일	CPF 에서 먼저 볼 것	상세
REST API 작성	@CpfController + DomainBaseController	Controller 작성
업무 로직 작성	@CpfService + DomainBaseService	Service 작성
DB 조회·저장	@CpfRepository/@CpfDao + 선택한 Persistence	DB·Paging
업무 TX 적용	@CpfTx	Transaction
조회 Cache	CpfCachePort / CpfCacheAsideService	Cache
공통 기준정보	CpfCode/Parameter/Message/Calendar Service	공통 기능
거래 추적·Audit	CpfContext + @CpfLogging/@CpfAudit	Logging
외부 시스템 호출	@CpfClient + Timeout/Retry + CpfServiceResult	외부 연계
다른 Domain 호출	Generated Typed Client + CpfResult	Domain 호출
메시지 수신	@CpfMessageListener + Idempotency	Messaging
권한·위험 작업	@CpfPermission / @CpfApprovalRequired	Security
테스트·검증	업무 Test + cpfVerifyFast	Test

먼저 기억할 원칙 업무 코드는 Framework 내부가 아니라 Generated Domain 의 Business Controller/Service/Repository 에 둡니다. CPF 가 제공하는 Base 와 Annotation 을 시작점으로 쓰고, Internal Package 를 직접 참조하지 않습니다.

Online 거래 개발 흐름

업무 개발자는 Controller → Service → Repository 계약을 사용하고, CPF가 Context·Transaction·오류·관측을 연결합니다.



핵심: 업무 로직은 Service에 두고, Remote/Async 호출은 Local DB Transaction과 별도 경계로 설계합니다.

업무 요청은 Controller → Service → Data/연계 경계를 따라 실행되고 Context·Transaction ID 로 추적됩니다.

2. 자주 쓰는 명령과 확인 위치

[전체 목차로](#)

일상 개발 명령은 `cpf-dev.ps1` 한 곳으로 모여 있습니다. 아래 Summary 에서 목적에 맞는 Action 을 고르고, 세부 옵션이 필요할 때만 도움말과 Specification 을 확인합니다.

```
pwsh .\cpf-tools\build\tools\cpf-dev.ps1 [Action] -ResourceProfile local
```

2.1 개발·실행

용도	명령어	세부 용도	주요 옵션	참고사항
개발 메뉴	(인자 없음)	번호 메뉴에서 작업 선택	-ResourceProfile	처음 쓰면 여기서 시작
로컬 실행	run-local	통합 WAS 1 개 / 1 Port 실행	local/dev/test/stg/prod	일반 Online 개발 기본
실행 상태	status	Local Runtime 기동 상태 확인	-	실행 전후 확인
실행 종료	stop	Local Runtime 종료	-	개발 종료 시 사용
Batch 실행	run-batch	Batch Runtime 만 별도 실행	-ResourceProfile	Batch 개발 때만 사용

2.2 빌드·검증

용도	명령어	세부 용도	주요 옵션	참고사항
전체 빌드	build	Compile 과 기본 품질 Gate 확인	-ResourceProfile	코드 변경 후 기본 확인
Java Test	test	전체 Java Test 만 실행	-ResourceProfile	Test 만 빠르게 볼 때
빠른 검증	verify-fast	계약·Catalog·정적 Gate 확인	-ResourceProfile	평소 기본 검증
전체 검증	verify-full	Java·Frontend·DB·Runtime 최대 범위	-ResourceProfile / -OutputRoot	마무리 검증

2.3 구성 확인

용도	명령어	세부 용도	주요 옵션	참고사항
Public 기능	modules	개발자가 선택할 Public Starter 확인	-ResourceProfile	Internal 기능은 숨김
자원 정책	resource	환경별 메모리·자원 정책 확인	-ResourceProfile	설정 위치와 함께 확인

환경값은 `gradle/cpf-runtime/common.properties` 와 `local/dev/test/stg/prod.properties` 에서 확인합니다.
일반 업무 개발자는 이 위치와 자신의 Domain 설정만 먼저 보면 됩니다.

3. Controller 작성

[전체 목차로](#)

3.1 기본 형태

Business Controller 에는 `@CpfController`를 붙이고 DomainBaseController를 상속합니다. `@CpfController`는 단순 `@RestController` 별칭이 아니라 CPF Context와 Base 구조를 Runtime에서 검증하는 진입점입니다.

```
@CpfController
@RequestMapping("/api/v1/member/samples")
public class SampleTransactionController extends MemberBaseController {
    private final SampleTransactionService service;

    public SampleTransactionController(SampleTransactionService service) {
        this.service = service;
    }

    @PostMapping
    public ResponseEntity<SampleItem> create(@Valid @RequestBody CreateSampleRequest request)
    {
        SampleItem item = service.create(request);
        return created(URI.create("/api/v1/member/samples/" + item.getSampleItemId()), item);
    }

    @GetMapping("/{id}")
    public ResponseEntity<SampleItem> detail(@PathVariable long id) {
        return ok(service.detail(id));
    }
}
```

3.2 Base에서 바로 쓰는 것

기능	메서드	업무에서 쓰는 때
현재 Context	requireCurrentContext()	transactionId/actor/tenant 등이 필요할 때
Context Snapshot	requireContext()	전달 가능한 Snapshot 이 필요할 때
입력 확인	requireText / requireValue / requireRule	단순 선행조건을 명확히 실패시킬 때
성공 응답	ok / created / noContent	표준 HTTP 성공 응답을 만들 때
실행 사실	executionFacts(operation)	Log/Audit에 비민감 식별자만 전달할 때

3.3 Header와 오류 응답

Header 문자열을 Controller마다 직접 파싱하기보다 `CpfContext` / `CpfHeaders`가 제공하는 값을 사용합니다. Transaction ID, Execution ID, Idempotency Key 같은 값은 공통 경계에서 정규화됩니다.

오류 응답은 직접 만들지 않기 업무 검증 실패는 CPF Exception으로 표현하고 `CpfGlobalExceptionHandler`가 HTTP 상태, 외부 메시지, transactionId, executionId를 표준 응답으로 만듭니다. SQL/Secret/Raw Exception을 응답에 넣지 않습니다.

4. Service 작성

[전체 목차로](#)

Business Service 에는 `@CpfService` 를 붙이고 DomainBaseService 를 상속합니다. Framework 의 `CpfBaseService` 는 최소 공통 동작만 제공하며, Domain 에서 반복되는 Context helper 가 필요하다면 Domain Base 에 둡니다.

```
@CpfService
public class SampleTransactionService extends MemberBaseService {
    private final SampleTransactionRepository repository;

    @CpfTx(id="MBR_CREATE_SAMPLE", name="MBR_SAMPLE_TX_CREATE", ownerDomain="MBR")
    public SampleItem create(CreateSampleRequest request) {
        String name = requireText(request.itemName(), "itemName");
        // 업무 검증 → Repository 호출 → 결과 반환
        return repository.insertAndFind(...);
    }
}
```

필요한 것	권장 위치	기억할 점
입력 정규화·필수값	Service 시작부	단순 값 검증은 빠르게 실패
업무 규칙	Service	Controller/Repository 에 흩어놓지 않음
Transaction	Service public method	업무 단위를 기준으로 @CpfTx 적용
다른 Service 호출	Service → Service	같은 TX 를 공유할지 먼저 판단
업무 오류	CpfBusinessException 등 CPF 오류	Controller 가 오류 Response 를 직접 조립하지 않음

Base Class 를 억지로 채우지 않기 `CpfBaseService` 의 공통 기능이 적은 것은 의도적입니다. 특정 Domain 에서만 쓰는 helper 를 Framework Base 에 올리지 말고 DomainBaseService 에 두십시오.

5. DB·Repository·Paging

[전체 목차로](#)

프로젝트에서 이미 선택한 Persistence(JDBC/MyBatis/JPA)를 그대로 사용합니다. 신규 개발자가 Provider 를 다시 고르는 것이 아니라, 해당 Provider 위에서 CPF Repository/DAO 경계를 맞추는 것이 핵심입니다.

현재 방식	잘 맞는 코드	업무 개발 포인트
JDBC	SQL + Row mapping	단순 SQL 과 명시적 Mapping
MyBatis	Mapper + SQL	복잡 조회·동적 SQL·기존 SQL 자산
JPA	Entity + Repository	Entity 관계·상태 중심 모델

5.1 Repository 기본 형태

```
@CpfRepository
public class SampleTransactionRepository extends MemberBaseRepository {
    private final SampleTransactionMapper mapper;

    public Optional<SampleItem> findById(long id) {
        requireRule(id > 0, "sampleItemId 는 1 이상이어야 합니다.");
        return Optional.ofNullable(mapper.findById(id));
    }

    public List<SampleItem> search(..., int page, int size, String sortBy, String direction) {
        int safeSize = pageSize(size);
        String sort = SORT_COLUMNS.contains(sortBy) ? sortBy : "sample_item_id";
        return mapper.search(..., pageOffset(page, safeSize), safeSize, sort, direction);
    }
}
```

5.2 CRUD·검색·Paging

필요	권장 방식	선택 기준·주의
단건 조회	Optional 또는 명확한 Not Found	null 성공으로 숨기지 않음
목록+총건수	Page	전체 페이지 수가 필요한 화면
다음 데이터 여부	Slice	count 비용을 줄이고 싶은 목록
연속 대량 조회	Cursor	안정적인 Sort Key 가 있을 때
동시 수정	Version 또는 findForUpdate	충돌을 409/업무 오류로 명시
Bulk	선택 Provider 의 batch API	처리 단위와 메모리 상한을 둠

CPF 공통 Paging 계약에는 `CpfPageRequest`, `CpfPage`, `CpfSlice`, Cursor 관련 타입이 있습니다. 실제 Domain DTO 규약이 별도로 정해져 있으면 그 계약을 우선하되, page/size 상한과 Sort allowlist 는 반드시 둡니다.

5.3 동시성·중복 요청

목적	CPF/업무 패턴	선택 기준	참고사항
동일 Row 충돌	Version / Optimistic Lock	일반적인 동시 수정 충돌	충돌을 명확한 업무 오류로 반환
DB 선점	findForUpdate 등 DB Lock	짧은 TX 안에서 직렬화가 꼭 필요	Remote 호출을 Lock 안에 오래 두지 않음
공유 자원 Lock	CpfDistributedLockPort	여러 Instance 가 같은 자원을 조정	wait/lease 와 소유권 해제 고려
중복 요청 방지	@CpfIdempotent	같은 업무 요청의 재전송 가능성	payload 충돌·진행 중·결과 재생까지 고려

Native escape hatch 표준 API 로 표현하기 어려운 Vendor/Provider 기능이 꼭 필요하다면 Repository/Adapter 경계에서 제한적으로 사용하고 Service/Controller 까지 Native 타입을 퍼뜨리지 않습니다.

6. Transaction

[전체 목차로](#)

일반 업무 Transaction 은 Service method 에 `@CpfTx` 를 붙이는 것이 기본입니다. `id`, `name`, `ownerDomain` 은 필수이고 `propagation`, `isolation`, `readOnly`, `timeoutSeconds` 등은 필요할 때만 조정합니다.

```
@CpfTx(
    id = "MBR_UPDATE_SAMPLE",
    name = "MBR_SAMPLE_TX_UPDATE",
    ownerDomain = "MBR",
    timeoutSeconds = 5
)
public SampleItem update(...) { ... }
```

선택	언제	주의
REQUIRED	일반적인 업무 Service	호출 중 기존 TX 가 있으면 참여
REQUIRES_NEW	감사/독립 기록처럼 반드시 별도 Commit 이 필요할 때	외부 업무 TX 실패와 독립적으로 남는 것이 맞는지 검토
readOnly=true	조회 전용 Service	쓰기 동작을 섞지 않음
timeoutSeconds	DB 대기 상한이 필요한 업무	Remote timeout 과 같은 값으로 착각하지 않음

- 같은 Class 내부에서 자기 `@CpfTx` method 를 직접 호출하는 self-invocation 은 Proxy 경계를 우회할 수 있으므로 TX 분리를 기대하지 않습니다.

- DB Transaction 과 HTTP/Message 같은 Remote Side Effect 를 하나의 원자 Transaction 으로 가정하지 않습니다.
- Rollback 후 외부 시스템이 이미 처리했을 수 있는 호출은 Idempotency/Reconcile 설계를 함께 둡니다.

7. Cache·Lock

[전체 목차로](#)

단순 get/put 이 필요하면 `CpfCachePort`, Cache Aside 가 필요하면 `CpfCacheAsideService` 를 사용합니다. Provider(Caffeine/Redis/Valkey) 구현을 업무 코드가 직접 알 필요는 없습니다.

하고 싶은 일	CPF API	기억할 점
조회	cache.get(key)	miss 와 장애를 구분
저장	cache.put(key,value,ttl)	TTL 을 업무 의미에 맞춤
단건 무효화	cache.evict(key)	변경 직후 stale 방지
Namespace 무효화	evictNamespace(...)	대량 무효화는 영향 범위 확인
Cache Aside	getOrLoad(key,options,loader)	single-flight/negative cache 포함
분산 동기화	CpfDistributedLockPort	정말 공유 Lock 이 필요한 업무에만

```
CpfCacheOptions options = new CpfCacheOptions(
    Duration.ofMinutes(10), Duration.ofSeconds(30),
    Duration.ofMillis(200), Duration.ofSeconds(5),
    true, true);
```

```
CpfCacheValue value = cacheAside.getOrLoad(key, options, () -> loadFromDb(key));
```

failOpen 의미 Cache 장애를 원본 조회의 실패처럼 만들지 않을지 결정하는 옵션입니다. 하지만 원본 Loader 자체가 실패한 것을 Cache miss 로 숨기지는 않습니다.

8. 공통코드·파라미터·메시지·영업일

[전체 목차로](#)

업무에서 자주 반복되는 기준정보는 각 Table 을 직접 조회하지 말고 `cpf-starter-common` 의 공개 Service 를 사용합니다.

필요	공개 Service	대표 사용
공통코드	CpfCodeService	values / find / required
업무 파라미터	CpfParameterService	find / requiredValue / typed value
메시지	CpfMessageService	locale/argument 를 반영한 메시지 해석
영업일	CpfCalendarService	isBusinessDay / next / previous / shift
Template	CpfTemplateService	업무 Template 을 정책에 맞게 조회·렌더

```
int maxRetry = parameterService.requiredValue("MAX_RETRY", Integer.class);
LocalDate next = calendarService.nextBusinessDay("DEFAULT", businessDate);
```

직접 SQL 반복 금지 공통코드·파라미터·메시지·영업일은 Cache/변경 반영 정책이 함께 있으므로 업무 Domain 에서 같은 Table 을 직접 조회하는 코드를 반복하지 않습니다.

9. Logging·Context·Audit

[전체 목차로](#)

업무 개발자는 “무엇을 남길지”만 명확히 하면 됩니다. Transaction ID/Execution ID 같은 상관관계는 CPF Context 가 제공하고, 민감정보는 allowlist 와 masking 정책을 따릅니다.

목적	사용	언제
일반 실행 로그	@CpfLogging	업무 method 의 안전한 요약 로그가 필요할 때
감사	@CpfAudit	변경/위험 행위와 사유·결과를 남길 때
상관관계	CpfContext / executionFacts	transactionId·executionId 연결
오류	CPF Exception + Global Handler	외부 메시지와 내부 원인을 분리

```
@CpfLogging(operation = "MEMBER_SEARCH")
public PageResult search(...) { ... }

@CpfAudit(action = "MEMBER_STATUS_CHANGE", reasonRequired = true)
public void changeStatus(...) { ... }
```

민감정보 Token, Password, Secret, 주민/계좌 등 민감 원문을 log argument/result 로 포함하지 않습니다. `includeArguments=true` 를 쓸 때도 allowlist 를 우선합니다.

10. 외부 연계

[전체 목차로](#)

외부 시스템 호출은 Transport 구현을 업무 코드가 직접 제어하기보다 CPF Integration 경계에 등록합니다. 호출 method/type 에 `@CpfClient`, 필요하면 `@CpfTimeout`, `@CpfRetry`를 적용합니다.

```
@CpfClient(system = "PARTNER", operation = "findCustomer")
@CpfTimeout(millis = 1500)
@CpfRetry(maxAttempts = 2, delayMillis = 100)
public CpfServiceResult<CustomerResponse> findCustomer(...) { ... }
```

결과	의미	업무 처리
SUCCESS	성공 확정	responseBody 사용
BUSINESS_FAILURE	상대 업무 거절	업무 오류로 처리
TECHNICAL_FAILURE	통신/기술 실패 확정	정책에 따라 실패/재시도
UNKNOWN	처리 여부를 확정 못함	자동 성공/재시도 금지, recoveryId 로 Reconcile

`CpfServiceResult.resultStatus()`와 `success/businessFailure/technicalFailure/unknown` helper 를 사용하고 문자열 status 비교를 새 업무 코드에 반복하지 않습니다.

11. 다른 Domain 호출

[전체 목차로](#)

다른 업무 Domain 은 Generated Typed Client 를 주입받아 호출합니다. Business Source 는 Local/Remote 배치 위치를 직접 분기하지 않고 Runtime Binding 이 `AUTO/LOCAL/REMOTE`를 결정합니다.

```
@CpfService
public class DomainDependencySampleService extends MemberBaseService {
    private final ExternalDomainClient external;

    public ExternalPingResponse ping(...) {
        CpfResult<ExternalPingResponse> result = external.execute(request);
        return result.fold(
            ok -> ok,
            fail -> { throw mapFailure(fail); },
            unknown -> { throw toReconcile(unknown); }
        );
    }
}
```

개발자가 신경 쓸 것 Local/Remote 자체보다 결과 의미를 신경 쓰십시오. `CpfResult`의 SUCCESS/BUSINESS_FAILURE/TECHNICAL_FAILURE/UNKNOWN 을 빠뜨리지 않고 처리하는 것이 핵심입니다.

12. Messaging

[전체 목차로](#)

Broker 종류(Kafka/JMS/IBM MQ/RabbitMQ)는 선택한 Starter 가 담당합니다. 업무 Listener 는 `@CpfMessageListener` 로 destination, consumerGroup, Idempotency 요구를 선언합니다.

```
@CpfMessageListener(destination = "member.changed", consumerGroup = "mbr-worker")
public void onMemberChanged(MemberChangedEvent event) {
    // 업무 처리 - 기본 idempotencyRequired=true
}
```

관심사	업무 코드에서 확인
발행	공개 Publisher/Bridge 를 통해 Event + key/context 전달
수신	@CpfMessageListener 와 업무 handler
중복	업무 key/Inbox/Idempotency 정책
실패	Retry/DLQ 와 업무 재처리를 구분
응답 유실	발행/처리 여부가 불명하면 UNKNOWN/Reconcile

Broker client(KafkaTemplate/JmsTemplate 등)를 Business Service 곳곳에서 직접 사용하지 않습니다. Provider 변경 시 업무 코드가 같이 흔들리지 않도록 공개 Messaging 경계를 사용합니다.

13. Security·권한·승인

[전체 목차로](#)

인증 방식 자체보다 업무 method 에 필요한 권한과 위험 조치를 선언하는 것이 개발자의 주 관심사입니다.

```
@CpfPermission({"MEMBER_WRITE"})
@CpfAudit(action = "MEMBER_UPDATE")
public void update(...) { ... }

@CpfApprovalRequired(action = "MEMBER_BULK_DISABLE", approvals = 1)
public void bulkDisable(String approvalId, String reason, ...) { ... }
```

필요	Annotation	사용 시점
권한	@CpfPermission	인증된 사용자가 특정 업무 권한을 가져야 할 때
감사	@CpfAudit	변경·중요 행위의 불변 이력이 필요할 때
사전 승인	@CpfApprovalRequired	위험 작업을 승인 후에만 실행할 때

14. Test·오류 확인

[전체 목차로](#)

업무 Test 는 “Controller 가 뜬다”보다 실제 업무 계약을 증명해야 합니다. 최신 Source 의 `cpf-member` Sample 은 Controller contract 와 Transaction rollback/idempotency/version 충돌 같은 경계를 검증하는 참고점입니다.

대상	최소 Test	실패 시 먼저 볼 것
Controller	Validation/200·201/오류 응답	CpfGlobalExceptionHandler + Context
Service	정상/업무 오류/TX rollback	@CpfTx 경계와 exception
Repository	CRUD/검색/정렬/Lock	SQL·Mapper·Sort allowlist
분산 호출	4 상태 결과 분기	UNKNOWN 누락 여부
Cache	hit/miss/fail-open/loader failure	Cache 장애와 origin 장애 구분

```
pwsh .\cpf-tools\build\tools\cpf-dev.ps1 test
pwsh .\cpf-tools\build\tools\cpf-dev.ps1 verify-fast
```

오류를 찾는 가장 빠른 순서 1) 응답의 transactionId 확보 → 2) 거래 로그/Trace 확인 → 3) CPF 오류 코드와 원인 경계 확인 → 4) 분산 호출이면 UNKNOWN 여부 확인 → 5) 재현 Test 추가.

14.1 업무 개발 마무리 체크

- Controller 입력 Validation 과 표준 오류 응답을 Test 했습니다.
- Service 업무 규칙과 `@CpfTx` rollback 경계를 Test 했습니다.
- 검색/Paging/정렬 상한과 동시 수정 충돌을 확인했습니다.
- Remote/Async 경계가 있으면 SUCCESS/업무실패/기술실패/UNKNOWN 을 모두 처리했습니다.
- 중복 요청 가능성이 있으면 Idempotency 정책을 확인했습니다.
- `test` 와 `verify-fast` 를 통과한 뒤 필요한 경우 `verify-full` 로 마무리 검증합니다.

15. 필요 기능을 추가할 때

[전체 목차로](#)

기본 셋업 이후 새로운 기능이 필요하면 Module 이름부터 찾지 말고 이용 목적을 먼저 고릅니다. `cpf-dev.ps1 modules` 로 Public 선택 대상을 확인한 뒤 아래 기준으로 선택합니다.

용도	선택 기능	주요 선택 기준	참고사항
일반 REST API	web-api / secure-api / bff	인증·브라우저 전용 BFF 여부	기본 Profile 중 선택
DB 접근	JDBC / MyBatis / JPA	SQL 중심인지 Entity 중심인지	프로젝트의 기존 방식을 우선
Cache	Caffeine / Redis / Valkey	단일 Instance 인지 공유 Cache 인지	공유 Cache 는 운영 Provider 확인
Messaging	Kafka / JMS / IBM MQ / RabbitMQ	조직 표준 Broker 와 연계 방식	Broker API 대신 CPF 경계 사용
로그인·Session	OIDC / JDBC Session / Valkey Session	로그인 방식과 Session 공유 필요	업무 권한은 @CpfPermission
분산 Lock	Valkey Lock	여러 Instance 가 같은 자원을 조정	DB Lock 과 목적을 구분
파일 저장	S3 Object Storage	S3 호환 저장소가 필요	업무 코드에 SDK 타입 확산 금지
기타 연계	GraphQL / Realtime	GraphQL Endpoint 또는 SSE 필요	필요한 경우에만 추가
업무 공통	cpf-starter-common	코드·파라미터·메시지·영업일	직접 SQL 반복 대신 공개 Service

정확한 Artifact ID 와 조합 규칙은 Starter Catalog/07 Specification 에서 확인합니다. Catalog 에서 internal 인 Module 은 Generated Domain 이 직접 의존하지 않습니다.

16. 빠른 Reference

[전체 목차로](#)

찾는 것	Source 시작점
Controller	cpf-starters/web/.../com/cpf/web/api
Service/Base	cpf-starters/base/runtime/.../foundation
TX/Repository	cpf-starters/data/persistence/.../api
Cache/Lock	cpf-starters/data/.../cache/api
공통 기능	cpf-starters/common/.../*api
Integration	cpf-starters/integration/.../api
Messaging	cpf-starters/messaging/.../api
Security/Audit	cpf-starters/security + platform-operations
실제 업무 예	cpf-member/online + cpf-external + cpf-education

더 정확한 계약이 필요하면 Method/속성/상태/Config의 정확한 Reference는 07 Specification을 사용합니다. 이 개발자 가이드는 “어떻게 업무에 적용하는지”에 집중합니다.